

1. Результаты SQL и независимый контроль

Для проверки корректности работы пайплайна был выполнен запрос к опубликованной витрине:

```
SELECT * FROM published.mart_daily_sales ORDER BY date, historical_region;
```

Ожидаемое значение (ручной расчет по исходным CSV):

2023-10-01, North: 150.00 (1 заказ)

2023-10-01, South: 200.00 (1 заказ)

2023-10-02, North: 50.00 (1 заказ)

2023-10-02, East: 300.00 (1 заказ)

Наблюдение: Результат выполнения SQL в PostgreSQL полностью совпал с ручным расчетом. Связь факта с измерением dim_customers по интервалам времени (valid_from / valid_to) отработала корректно.

2. Таблица четырёх диагностических случаев

В проекте настроены тесты dbt (уникальность, заполненность, связи, бизнес-правила). Ниже приведены сценарии контролируемых изменений входа.

№	Что изменить (Вход)	Прогноз	Наблюдение	Объяснение
1	Изменить amount на -150 в raw_orders.csv	Пайплайн упадет на dbt_test. Витрина не обновится.	Задача dbt_test стала красной (FAIL 1). В схеме published остались старые данные. (Подтверждено на скриншотах.)	Сработал кастомный SQL-тест test_positive_amount, блокирующий отрицательную выручку.
2	Добавить заказ с несуществующим customer_id=999	Пайплайн упадет на dbt_test.	Задача dbt_test падает с ошибкой relationships.	Сработал встроенный тест dbt на целостность внешних ключей (Fact -> Dimension).
3	Продублировать строку с order_id=101 в CSV	Пайплайн упадет на dbt_test.	Задача dbt_test падает с ошибкой unique.	Сработал тест на уникальность первичного ключа факта.
4	Добавить в raw_customers.csv запись для Alice с пересекающимся updated_at	Пайплайн упадет на dbt_test.	Задача dbt_test падает, тест assert_no_scd_overlap возвращает строки.	Сработал сингулярный тест, проверяющий, что интервалы жизни SCD2-записей не пересекаются.

3. Сравнение повторов и ограничения решения

Повторный расчет: Двойной запуск DAG на одинаковом входе дает идентичный результат (идемпотентность). Это достигается за счет того, что скрипт extractor.py выполняет TRUNCATE таблиц перед загрузкой, а dbt использует конструкцию CREATE OR REPLACE / DROP + CREATE для таблиц.

Параллельные запуски: Конфликт исключен за счет использования SequentialExecutor в Airflow и настройки `max_active_runs=1`.

Ограничение проверок: Тест на уникальность (`unique`) гарантирует отсутствие дублей в DWH, но не доказывает полноту выгрузки. Если источник (API) потеряет часть заказов и не пришлет их, dbt этого не заметит, так как не знает об ожидаемом общем количестве записей.